How To Become A Mechanistic Interpretability Researcher

by **Neel Nanda** 2nd Sep 2025

Last updated Sept 2 2025

Note - if you want to pursue a career in this kind of research, apply to my MATS stream! Due Dec 23

TL;DR

- This post is about the mindset and process I recommend if you want to *do* mechanistic interpretability research. I aim to give a clear sense of direction, so give opinionated advice and concrete recommendations.
  - Mech interp is high-leverage, impactful, and learnable on your own with short feedback loops and modest compute.
  - **Learn the minimum viable basics, then do research.** Mech interp is an empirical science
- Three stages:
  - **Learn the ropes** **(≤1 month)** learn the essentials, go breadth-first;
  - **Learn with research mini-projects** practice basic research skills with 1-5 day mini projects, focus on fast feedback loop skills;
  - **Work up to full projects**, do 1-2 week research sprints, continue the best ones. Explore deeper skills and the mindset of a great researcher.
- **Stage 1:** **Learning the Ropes**
  - **Breadth over depth; get a good baseline not perfection**

    - **Learn the basics**: Code a transformer from scratch°, key mech interp techniques °, the landscape of the field°, linear algebra intuitions°, how to write mech interp code° (ARENA is your friend)
    - **Get your hands dirty**: Do *not* just read things. Mech interp is a fundamentally empirical science
    - **Move on after a month**. Don't expect to feel "done" or to have covered *all* of the ropes, learn more when needed. You won't stumble across great research insights without starting to do something real
    - **Use LLMs extensively°** - they're not perfect, but are better at mech interp than you right now! They're a crucial learning tool (when used right!)
- **Unpacking the research process°**:
    - Many skills°, categorise them by the feedback loops.
        - Fast skills (minutes-hours) like write/run/debug experiments
        - Slow (weeks) like how to prioritise and when to pivot
        - Very slow (months) like generating good research ideas
    - **Do *not* try to learn all skills at once**. Focus on fast/medium skills first, then slowly expand
    - 4 phases of research°: finding an idea (**ideation**) -> building intuition and hunches (**exploration**) -> testing hypotheses (**understanding**) -> refining and writing up (**distillation**)
- **Stage 2:° Mini projects** (1-5 days each for 2-4 weeks)
    - Exploration mindset°: **Maximise information gain per unit time**, learn how to get unstuck. You don't need a plan, so long as you're learning
    - Understanding mindset°: **Every research result is false until proven otherwise**. The more exciting a result is, the more likely it is to be false. Be your own greatest critic
    - Idea quality (ideation) and write-ups (distillation) aren't the priority yet; **taste and prioritization are learned by doing things**.
    - Having good research ideas takes forever to learn, **to choose early projects, cheat**! Pick well scoped projects°, eg extending a paper (ideas)
    - **Use LLMs extensively°** - they should speed up your research/coding a *lot* (if you know how to use them properly!)
- **Stage 3:° Towards full projects**

    - **Work in 1-2 week sprints**, post-mortem after each, pivot to another project unless it's going *great*
    - **Slower skills° and key mindsets°**: careful skepticism, awareness of the literature, prioritization, high productivity
    - **Do good science°, not flashy science** - be honest about limitations, give proof you're not cherry picking, read your data, do the simple things that work, use real baselines.
    - **Write-up° your work**! Distill it into a narrative, then iteratively expand it to a write-up
        - **Good public work is your best credential°** - for careers, PhDs, finding mentors, etc
        - **Writing is not an afterthought** - make time for it. The reader will understand less than you think°
    - **Practice generating research ideas°**. If possible, try to imitation learn a mentor's research taste.°
        - Avoid fads°, and think about what's new and exciting in mech interp°
- **Proactively reach out to mentors°** Everything is *much* easier with a good mentor. Cold email, apply for mentoring programs, etc.
    - Reach out to researchers who'll have time, not the most famous
- **Careers:** If you want to work in the field, apply for things! Jobs°, mentoring programs°, funding°, academic labs°.
    - Bonus thoughts: what do hiring managers look for°, what does a good research mentor actually do°, and should you do a PhD°?
- I also give various thoughts on how I'm thinking about the field nowadays, and what I've changed my mind about. I separate these from the practical advice, so you can take it or leave it.
    - NEW (Dec 1 25): See related posts on the much more pragmatic approach I am taking to research and research directions we think are promising
    - Covering: how I currently define the field°, why I'm pessimistic on ambitious reverse engineering, and excited about more pragmatic approaches°, what recent work I am excited about° and recommend building on.

Introduction

Mechanistic interpretability (mech interp) is, in my incredibly biased opinion, one of the most exciting research areas out there. We have these incredibly complex AI models that we don't understand, yet there are tantalizing signs of real structure inside them. Even partial understanding of this structure opens up a world of possibilities, yet is neglected by 99% of machine learning researchers. There's so much to do!

I think mech interp is an unusually easy field to learn about on your own: there's a lot of educational materials, you don't need too much compute, and there's short feedback loops. But if you're new, it can feel pretty intimidating to get started. This is my updated guide on how to skill up, get involved, reach the point where you can do actual research, and some advice on how to go from there to a career/academic role in the field.

This guide is deliberately highly opinionated. My goal is to convey a productive mindset and concrete steps that I think will work well, and give a sense of direction, rather than trying to give a fully broad overview or perfect advice. (And many of the links are to my own work because that's what I know best. Sorry!)

High-Level Framing

My core philosophy for getting into mech interp is this: learn the absolute minimal basics as quickly as possible, and then immediately transition to learning by doing research.

The goal is not to read every paper before you touch research. When doing research you'll notice gaps and go back to learn more. But being grounded in a project will give you vastly more direction to guide your learning, and contextualise why anything you're learning actually matters. You just want enough grounding to start a project with some understanding of what you're doing.

Don't stress about the research quality at first, or having the perfect project idea. Key skills, like research taste and the ability to prioritize, take time to develop. Gaining experience—even messy experience—will teach you the basics like how to run and interpret experiments, which in turn help you learn the high-level skills.

I break this down into three stages:

1. **Learning the ropes**, where you work through the basics breadth first, and after at most a month, move on to stage 2
2. **Practicing research with mini-projects**. Work on throwaway, 1-5 day research projects. Focus on practicing the basic research skills with the fastest feedback loops,

don't stress about having the best ideas, or writing them up. After 2-4 weeks, move on to stage 3

3. **Work up to full-projects**°: work in 1-2 week sprints. After each, do a post-mortem and pivot to something else, *unless* it was going great and has momentum. Eventually, you should end up working on something longer-term. Start thinking about the deeper skills and research mindsets, practice having good ideas, and prioritize making good public write-ups of sprints that went well

Stage 1: Learning the Ropes

Your goal here is learning the basics: how to write experiments with a mech interp library, understanding the key concepts, getting the lay of the land.

Your aim is learning enough that the rest of your learning can be done via doing research, *not* finishing learning. Prioritize ruthlessly. **After max 1 month**[1]**, move on to stage 2**. I've flagged which parts of this I think are essential, vs just nice to have.

Do not just read papers - a common mistake among academic types is to spend months reading as many papers as they can get their hands on before writing code. Don't do it. Mech interp is an empirical science, getting your hands dirty gives key context for your learning. Intersperse reading papers with doing coding tutorials or small research explorations. See my research walkthroughs for an idea of what tiny exploratory projects can look like.

LLMs are a key tool - see the section below° for advice on using them well

Machine Learning & Transformer Basics

Assuming you already know basic Python and introductory ML concepts.

- **Maths:**
  - **Linear Algebra is King (Essential):** You need to think in vectors and matrices fluently. This is by far the highest value set of generic math you should learn to do mech interp or ML research.
    - *Resource:* 3Blue1Brown's Essence of Linear Algebra.
    - **Highly recommended**: Put A Mathematical Framework For Transformer Circuits in the context window and have the LLM generate exercises to test your intuitions about transformer internals.

        - LLMs are great for checking whether linear algebra actually clicks. Try summarizing what you've learned and the links between different concepts and ask an LLM whether you are correct. For example:
            - Ensure you understand SVD and why it works
            - What does changing basis mean and why does it matter
            - Key ways a low rank and full rank matrix differ
    - **Other Bits:** Basic probability, info theory, optimization, vector calculus.
        - Use an LLM tutor to quiz your understanding on the parts most relevant to transformers
    - Generally don't bother learning other areas of maths (unless doing it for fun!)
- **Practical ML with PyTorch: (Essential)**
    - Code a simple Transformer (like GPT-2) from scratch. ARENA Chapter 1.1 is a great coding tutorial[2]
        - This builds intuitions for mech interp *and* on using PyTorch.
        - I have two video tutorials on this, starting from the basics - start here if you're not sure what to do!
        - And use LLMs to fill in any background things you're missing, like PyTorch basics
- **Cloud GPUs:**
    - You'll need to be able to run language models, which (typically) needs a GPU
    - You can start with Google Colab to get started fast, but it'll be very constraining to use long-term. Learn to rent and use a cloud GPU.
        - Newer Macbook Pros, or computers with powerful gaming GPUs may also be able to run LLMs locally
    - *Resource:* ARENA has a guide. I like runpod.io as a provider; vast.ai is cheaper.
    - nnsight also lets you do some interpretability on certain models they host themselves, including LLaMA 3 405B, which can be a great way to work with larger models.

Mechanistic Interpretability Techniques

A lot of mech interp research looks like knowing the right technique to apply and in what context. This is a key thing to prioritise getting your head around when starting out. You'll

learn this with a mix of reading educational materials and doing coding tutorials like ARENA (discussed in next sub-section).

- Ferrando et al is a good **overview** of the key techniques - it's long enough that you shouldn't prioritise reading it in full, but it's a great reference
  - Put it in a LLM context window and ask questions, or to write you exercises
- **Essential**: Make sure you understand these **core techniques**, well enough that you can code it up yourself on a simple model like GPT-2 Small[3]:
  - Activation Patching
  - Linear Probes
  - Using Sparse Autoencoders (SAEs) (you only need to write code that uses an SAE, not trains one)
  - Max Activating Dataset Examples
  - Nice-to-have:
    - Steering Vectors
    - Direct Logit Attribution (DLA) (a simpler version is called logit lens)
  - **Key exercise**: Describe each technique to an LLM with Ferrando et al in the context window and ask for feedback. Iterate until you get it all right.
    - Use an anti-sycophancy prompt to get real feedback, by pretending someone else wrote your answer, e.g. "I saw someone claim this, it seems pretty off to me, can you help me give them direct but constructive feedback on what they missed? [insert your description]"
- Remember that there's a bunch of valuable **black-box interpretability** techniques! (ie that don't use the model's internals) You can often correctly guess a model's algorithm by reading its chain of thought. Careful variation of the prompt is a powerful way to causally test hypotheses.
  - They're an additional tool. Often the correct first step in an investigation is just talking to the model and bunch and observing its behaviour. Don't be a purist and dismiss them as "not rigorous" - they have uses and flaws, just like any other technique.
    - One project I supervised° on interpreting "self-preservation" in frontier models started with simple black-box techniques, and it just worked, we never needed anything fancier.

- Understand fancier black-box techniques like token forcing (aka prefill attacks) where you put words in a model's mouth.

Mechanistic Interpretability Coding & Tooling

- **Goal:** Get comfortable running experiments and "playing" with model internals. Get the engineering basics down[4]. Get your hands dirty**.**
- **ARENA**: ARENA has a set of fantastic coding tutorials by Callum McDougall, you should just go do these. But there's tons, so **prioritize ruthlessly**.
  - **Essential*:* Chapter 1.2** (Interpretability Basics – prioritize the first 3 sections on tooling, direct observation, and patching).
  - *Recommended:* 1.4.1 (Causal Interventions & Activation Patching – this is a core technique).
  - *Worthwhile*: 1.3.2 (Sparse Autoencoders (SAEs) – Skim or Skip section 1, the key thing to get from the rest is an intuition for what SAEs are, strengths and weaknesses, and how to use an open source SAE. Don't worry about training them).
- **Tooling** (**Essential**)**:** Get proficient with at least one mech interp library, this is what you'll use to run experiments.
  - TransformerLens: best for small models <=9B where you want to write more complex interpretability experiments, or work with many models at once.
    - As of early Sept 2025, TransformerLens v3 is in alpha, works well with large models and is far more flexible.
  - nnsight: More performant, works well on larger models, it's just a wrapper around standard LLM libraries like HuggingFace transformers
- **LLM APIs**: Learn how to use an LLM API to call an LLM programmatically. This is super useful for measuring qualitative things about some data, and for generating synthetic datasets
  - I like openrouter.ai which lets you access almost all the important LLMs from a single place. GPT5 and Gemini are reasonably priced and good defaults, they have a range of sizes
    - Cerebras and Groq have *way* higher throughput than normal providers, and serve a handful of open source models, they may be worth checking out.
  - Exercise: Make a happiness steering vector (for e.g. GPT-2 Small) by having an LLM via an API generate 32 happy prompts and 32 sad prompts, and taking the difference in mean activations[5] (e.g. the residual stream at the middle layer).

Add this vector to the model's residual stream[6] while generating responses to some example prompts, and use an LLM API to rate how happy they seem, and see this score go up when steering.

- **Open source LLMs**: You'll want to work a lot with open source LLMs, as the thing you're trying to interpret. The best open source LLM changes a lot
  - As of early Sept 2025, Qwen3 is a good default model family. Each model has reasoning and non-reasoning mode, there's a good range of sizes, and most are dense[7]
    - Gemma 3 and LLaMA 3.3 are decent non-reasoning models. I've heard bad things about gpt-oss and LLaMA 4
  - *Gotcha:* The different open source LLMs often have different tokenizations and formats for chat or reasoning tokens. Using the wrong token format can only somewhat degrade performance and may be hard to notice while corrupting your results - keep an eye out, try hard to find where this might be documented, and sanity check by e.g. comparing to official evals

Understanding the literature

Your priority is to understand the concepts and the basics, but you want a sense for the landscape of the field, so you should practice reading at least some papers.

- Remember, **breadth over depth**. Skim things, get a sense of what's out there, and only dive into the things that are most interesting.
  - You should be heavily using **LLMs** here. Give them something you're considering reading and get a summary, ask questions about the work, summarise your understanding to it and ask for feedback (with an anti-sycophancy prompt).
    - If you aren't able to verify yourself, cross-reference by asking multiple LLMs and making sure they all say consistent things.
- Here's a list of my favourite papers° (as of mid 2024) with summaries and opinions
  - Do *not* try to read all of these in full. Skim summaries, skim abstracts, pick a few to explore deeper with an LLM, *then* decide if you want to read the full paper.
  - My YouTube Channel: Paper walkthroughs, recordings of myself doing research, and talks.
- Open Problems In Mechanistic Interpretability is a decent recent literature review, that a lot of top mech interp people were involved in

    - Be warned that the paper basically consists of a bunch of opinionated and disagreeable researchers writing their own sections and often having strong takes. Don't defer to it too much, but it's a good way to quickly assess what's out there.
- **Deep dives**: You should read at least one paper carefully and in full. This is a useful skill that you will use in research projects where there's a handful of extremely relevant papers to your project
    - This is much more than just reading the words! You should write out a summary, try to understand the surrounding context with LLM help, be able to describe why the paper exists, the motivation, the problem it's trying to solve, etc.
    - Aim for a barbell strategy: put minimal effort into most papers and a lot of effort into a few.
- **LLMs**: LLMs are a super useful tool for exploring the literature, but easy to shoot yourself in the foot with.
    - As a search engine over the literature (especially with some lit reviews in context, or a starting paper), basically doing a lit review, finding relevant work for a question you have, etc.
        - As a tool to help you skim a paper - put the paper in the context window[8] then get a summary, ask it questions, etc
        - If you're concerned about hallucinations, you can ask it to support answers with quotes (and verify these are real and make sense), or give its answer to another LLM and ask for harsh critique of all the inaccuracies. Honestly, I often don't bother though, frontier reasoning models are pretty good now.
    - As a tool to help with deep dives - you need to actually read the paper, but I recommend having the LLM chat open as you read with the paper in the context and asking it questions, for context, etc every time you get confused.

Using LLMs for Learning

Note: I expect this section to go out of date fast! Written early Sept 2025

LLMs are a super useful tool for learning, especially in a new field. While they struggle to beat experts, they often beat novices. If you aren't using them regularly throughout this process, I'd guess you're leaving a bunch of value on the table.

But LLMs have weird flaws and strengths, and it's worth being intentional about how you use them:

- **Use a good model**: The best paid models are way better than e.g. free ChatGPT. Don't be a cheapskate; if you can, get a $20/month subscription, it makes a big difference. Gemini 2.5 Pro, Claude 4.1 Opus with extended thinking, and GPT-5 Thinking are all reasonable. (do *not* use non-thinking GPT-5 or anything older like GPT-4o, reasoning models are a big upgrade)
  - If you can't get a subscription, Gemini 2.5 Pro is also available for free, and is the best.
  - Use Gemini 2.5 Pro via AI Studio, it's way better than the main Gemini interface and has much nicer rate limits for free users. Always use compare mode (the button in the header with two arrows) to see two responses in parallel from Pro
  - See thoughts° from my MATS alum Paul Bogdan comparing different LLMs for learning, and why he currently prefers Gemini
- **System Prompts:** System prompts make a big difference - be concrete and specific about what you want, and how you want it done.
  - LLMs are good at this: I'll just ramble at one about what the task is, my criteria, the failure modes I don't want, and then it'll just write the prompt for me
  - If the prompt doesn't work, tell the LLM what it did wrong, and see if it can rewrite the prompt for you.
- **Merge perspectives**:
  - Ask a Q to multiple different frontier LLMs, give LLM B's response to LLM A and ask it to assess the strengths and weaknesses then merge.
    - If a point is in both original responses, it's probably not a hallucination
  - If you want to fact check an LLM's answer, give it to another LLM with an anti-sycophancy prompt
- **Anti-Sycophancy Prompts:** LLMs are bad at giving critical feedback. Frame your request so the sycophantic thing to do is to be critical, by pretending someone else wrote the thing you want feedback on.
  - *"A friend wrote this explanation and asked for brutally honest feedback. They'll be offended if I hold back. Please help me give them the most useful feedback."*
  - *"I saw someone claiming this, but it seems pretty dumb to me. What do you think?"*
  - *"Some moron wrote this thing, and I find this really annoying. Please write me a brutal but truthful response"*
- **Learn actively, not passively:**

    - **Summarize** your understanding back to the LLM in your own words and ask for critical feedback. Do this every time you read a paper or learn about a new concept
    - Try having it teach you **socratically**. Note: you can probably design a better system prompt than the official "study mode"
    - Ask the LLM to **generate exercises** to test your understanding, including maths and coding exercises as appropriate.
        - Gemini can make multiple choice quizzes, which some enjoy
        - Coding exercises can be requested with accompanying tests, and template code with blank functions for you to fill out, a la the ARENA tutorials.
- **Context engineering:** Modern LLMs are much more useful with relevant info in context. If you give them the paper in question, or source code of the relevant library[9], they'll be far more helpful.
    - See this folder for a bunch of saved context files for mech interp queries. If you don't know what you need, just use this default file.
    - I recommend Gemini 2.5 Pro (1M context window) via aistudio.google.com; the UI is better. Always turn compare mode on, you get two answers in parallel
- **Voice dictation**: If you dictate to your LLM, via free speech-to-text software, and run it with no editing, it'll understand fine. I personally find this much easier, especially when brain-dumping.
    - Superwhisper on Mac is great; Superwhisper is not currently available on Windows, but Windows users can use Whispr Flow.
- **Coding**: LLM tools like Cursor are great for coding, but *not* if your goal is to learn. For things like ARENA, only let yourself use browser-based LLMs, and only use them as a tutor. Don't copy and paste code, your goal is to learn not complete exercises.

Interlude: What is mech interp?

Feel free to skip to the "what should I do next°" part

At this point it's worth reflecting on what mech interp actually *is*. What are we even doing here? There isn't a consensus definition on how exactly to define mechanistic interpretability, and different researchers will give very different takes. But *my* working definition is as follows[10].

- **Interpretability** is the study of understanding models, gaining insight into their behavior, the cognition inside of them, why and how they work, etc. This is the important part and the heart of the field.
- **Mechanistic** means using the internals of the model, the weights and activations
- So **mechanistic interpretability** is any approach to understanding the model that uses its internals.
    - This is distinct from some other worthwhile directions, like **black box interpretability**, understanding models without using the internals, and **model internals**, using the internals of the model for other things like steering vectors.

Why this definition? To do impactful research, it's often good to find the directions that other people are missing. I think of most of machine learning as non-mechanistic non-interpretability. 99% of ML research just looks at the inputs and outputs to models, and treats its north star as controlling their behavior. Progress is defined by making a number go up, not to explain why it works. This has been very successful, but IMO leaves a lot of value on the table. Mechanistic interpretability is about doing better than this, and has achieved a bunch of cool stuff, like teaching grandmasters how to play chess better by interpreting AlphaZero.

Why care? Obviously, our goal is not "do things if and only if they fit the above definition", but I find it a useful one. To discuss this, let's first consider our actual goals here. To me, **the ultimate goal is to make human-level AI systems (or beyond) safer**. I do mech interp because I think we'll find enough understanding of what happens inside a model to be pragmatically useful here (also, because mech interp is fun!): to better understand how they work, detect if they're lying to us, detect and diagnose unexpected failure modes, etc. But people's goals vary, e.g. real-world usefulness today, aesthetic beauty, or scientific insight. It's worth thinking about what yours are.

Some implications of this framing worth laying out:

- My ultimate **north star is pragmatism** - achieve enough understanding to be (reliably) useful. Subgoals like "completely reverse engineer the model" are just means to an end.
    - One of my big shifts in research prioritization in recent years is concluding that **reverse engineering is not the right aim**. Instead, I think we should just more directly try to do pragmatic work that enables us to do useful things using internals. I discuss this shift more later on°.

- This is a **broad definition**. Historically, the field has focused on more specific agendas, like ambitious reverse engineering of models. But I think we shouldn't limit ourselves, there's many other important and neglected directions and the field is large enough to cover a lot of ground[11]
- It's about **understanding**, not just using internals - model internals methods like steering vectors can be useful for shaping a model's behaviour, but compete with many powerful methods like prompting and fine-tuning. Very few areas of ML can achieve understanding
- **Don't be a purist** - using internals is a means to an end. If black-box methods are the right tool, use them

The Big Picture: Learning the Craft of Research

So, you've gone through the tutorials, you understand the core concepts, and you can write some basic experimental code. Now comes the hard part: learning how to actually do mech interp research[12].

This is an inherently difficult thing to learn, of course. But IMO people often misunderstand what they need to do here, try to learn everything at once, or more generally make life unnecessarily hard for themselves. The key is to break the process down, understand the different skills involved, and focus on **learning the pieces with the fastest feedback loops first**.

I suggest breaking this down into two stages[13].

Stage 2: working on a bunch of throwaway mini projects of 1-5 days each. Don't stress about choosing the best projects or producing public output. The goal is to learn the skills with the fastest feedback loops.

Stage 3: After a few weeks of these, start to be more ambitious: paying more attention to how you choose your projects, gaining the subtler skills, and how to write things up. I still recommend working iteratively, in one to two week sprints, but ending up with longer-term projects if things go well.

Note: Unlike stage 1 to 2, the transition from stages two to three should be fairly gradual as you take on larger projects and become more ambitious. A good default would be after three to four weeks in stage two, but you don't need to have a big formal shift.

Mentorship: A good mentor is a major accelerator, and finding one should be a major priority for you. In the careers section, I provide advice on how to go about finding a good mentor°, and how concretely they can add value°. In the rest of the post I'll write most of it assuming you do not have a mentor and then flag the ways to use a mentor where appropriate.

Unpacking the Research Process

I find it helpful to think of research as a cycle of four distinct stages. Read my blog post on the research proces° for full details, but in brief:

- **Ideation:** You choose a research problem or a general domain to focus on.
- **Exploration:** You may not have a specific hypothesis yet; you're just trying to figure out the right questions to ask, and build deeper intuition for the domain. Your north star is to gain information and surface area.
- **Understanding:** This begins when you have a concrete hypothesis, and some intuitive understanding of the domain. Your north star is to convince yourself that the hypothesis is true or false.
- **Distillation:** Once you're convinced, your north star is to compress your findings into concise, rigorous truth that you can communicate to the world - create enough experimental evidence to convince others, write it up clearly, and share it.

Underpinning these stages is a host of skills, best separated by how quickly you can apply them and get feedback. We learn by doing things and getting feedback, so you'll learn the fast ones much more quickly. I put a rough list and categorization below.

My general advice is **to prioritize learning these in order of feedback loops**. If it seems like you need a slow skill to get started, like the taste to choose a good research problem, find a way to cheat rather than stressing about not having that skill (e.g. doing an incremental extension to a paper, getting one from a mentor, etc).

- **Fast Loop (minutes-hours):**
  - Planning and writing experiment code
    - **Medium**: Designing great experiments
    - **Medium**: Knowing when to write hacky vs. quality code.
  - Running/debugging experiments
    - **Medium/Slow**: Spotting and fixing subtle bugs (e.g., you got your tokenization subtly wrong, you didn't search hyper-parameters well enough,

etc)

    - Interpreting the results of a single experiment.
        - **Medium**: Understanding whether your results support your conclusions
        - **Slow**: Spotting subtle interpretability illusions where your results don't actually support your claims
- **Medium Loop (days):**
    - Developing a conceptual understanding of mech interp
        - **Slow**: Noticing and fixing your own subtle confusions
        - **Slow**: Build a deep knowledge of the literature
    - Knowing how to explore without getting stuck
    - Writing up results
        - **Slow**: Communicating your work in a way that's genuinely clear to people.
        - **Slow**: Communicating why your work is *interesting* to people
- **Slow Loop (weeks):**
    - Prioritizing which experiment to do next
    - Knowing when to continue with a research direction or pivot to another angle of attack/another project
    - Identifying bad research ideas, *without* doing a project on them first
- **Very Slow Loop (months):**
    - Coming up with good research ideas. This is the core of "research taste."

Your progression should be simple: First, focus on the fast/medium skills behind exploration and understanding with throwaway projects. Then, graduate to end-to-end projects where you can intentionally practice the deeper skills, and practice ideation and distillation too.

What is research taste?

A particularly important and fuzzy type of skill is called research taste. I basically think of this as the bundle of intuitions you get with enough research experience that let you do things like come up with good ideas, predict if an idea is promising, have conviction in good research directions, etc. Check out my post on the topic° for more thoughts.

I broadly think you should just ignore it for now, find ways to compensate for not having much yet, and focus on learning the fast-medium skills, and this will give you a much better base for learning it. In particular, it's much faster to learn with a mentor, so if you don't have a mentor at the start, you should prioritize other things.

But you want to learn it eventually, so it's good to be mindful of it throughout, and look for opportunities to practice and learn lessons. I recommend treating it as a nice-to-have but not stressing about it

Note, one important trap here is that having good taste can often manifest as having confidence and conviction in some research direction. But often novice researchers develop this confidence and conviction significantly *before* they develop the ability to not be confident in bad ideas. It's often a good learning experience to once or twice pursue a thing you feel really convinced is going to be epic and then discover you're wrong, so it's not that bad an outcome, especially in stage 2 (mini-projects) but be warned.

Stage 2: Practicing Research with Mini-Projects

With that big picture in mind, let's get our hands dirty. You want to do a series of ~1-5 day mini-projects, for maybe 2-4 weeks. The goal right now is to learn the craft, not to produce groundbreaking research.

Focus on practicing exploration and understanding and gaining the fast/medium skills, leave aside ideation and distillation for now. If you produce something cool and want to write it up, great! But that's a nice-to-have, not a priority.

Once you finish a mini-project, remember to do a post-mortem. Spend at least an hour analyzing: what did you do? What did you try? What worked? What didn't? What mistakes did you make? What would you do differently if doing this again? And how can you integrate this into your research strategy going forwards?

Choose A Project

Some suggested starter projects

- **Replicate and Extend a Paper:** A classic for a reason. Replicate a key result, then extend it. Suggestions:
  - Refusal is mediated by a single direction
    - Extending papers can vary a lot in difficulty. For example, applying the method to study refusal on a new model is easy as you can reuse the same data, while applying it to a new concept is harder.
    - Skills: practicing activation patching and steering vectors.

    - Thought Anchors: apply these reasoning model interpretability methods to new types of prompts, or explore some prompts using the linked interface, or see if you can improve on the methods/invent your own.
        - Skills: reasoning model interpretability, using LLM APIs, and working with modern models
    - Replicate the truth probes in Geometry of Truth on a more modern model and try applying them in more interesting settings. How well do they generalise? Can you break them? If so, can you fix this?
        - Skills: probing, supervised learning, dataset creation
- **Play around with something interesting:**
    - Use Neuronpedia's attribution graphs° to form a hypothesis about Gemma 2B, then use other methods (e.g. prompting) to verify it.
        - Skills: Attribution graphs, scientific mindset, prompting
    - Play with Bartosz Cywiński's taboo models that have a secret word programmed in and test as many methods as you can to find it.
        - If you're feeling ambitious: train your own models with a more complex secret, and try to interpret those.
        - Skills: Logit lens, SAEs, black box methods
    - Explore the models from the emergent misalignment° papers.
        - Skills: steering vectors, SAEs, maybe fine-tuning
    - Pick some prompts from Chain-of-Thought Reasoning In The Wild Is Not Always Faithful and try to gain a deeper understanding of what's happening
        - Skills: Open ended exploration, using whichever tools seem appropriate

Those cover two kinds of starter projects:

- **Understanding-heavy**, where you take a well-known domain and try to test a hypothesis there (e.g. extending a paper you've read closely)
    - Note that you still want to do *some*
- **Exploration-heavy**, where you take some phenomena (a technique, a model, a phenomena, etc) play around with it, and try to understand what's going on.
    - Exploration-heavy projects are often a less familiar style, so make sure to do some of those!

Common mistakes:

- People often get hung up on finding the "best" project. Sadly, that's not going to happen. Instead, just do something and see what happens - better ideas and inspiration come with time.
- Don't get too attached to your first project. It was probably badly chosen! These are throwaway projects, just move on once you're not learning as much.
- Conversely, don't flit between ideas so much that you never build your "getting unstuck" toolkit.
- Avoid compute-heavy and/papers (e.g., training cross-layer transcoders) or highly technical papers (e.g., Sparse Feature Circuits).

Practicing Exploration

The idea of exploration as a phase in itself often trips up people new to mech interp. They feel like they always need to have a plan, a clear thing they're doing at any given point, etc. In my experience, you will often spend more than half of a project trying to figure out what the hell is happening and what you think your plan is. This is totally fine!

You don't need a plan. It's okay to be confused. However, this does *not* mean you should just screw around. Your North Star: gain information and surface area[14] on the problem. Your job is to take actions that maximise information gained per unit time. If you've learned nothing in 2 hours, pivot to another approach. If 2-3 approaches were dead ends, it's fine to just pick another problem.

I have several research walkthroughs on my YouTube channel that I think demonstrates the mindset of exploration. What I think is an appropriate speed to be moving. E.g. I think you should aim to make a new plot every few minutes (or faster!) if experiments don't take too long to run.

A common difficulty is feeling "stuck" and not knowing what to do. IMO, this is largely a skill issue. Here's my recommended toolkit when this happens:

- Use "gain surface area" techniques, things that can surface new ideas and connections and just give you raw data to work with: look at the model's output/chain-of-thought, change the prompt, probe for a concept, look at an SAE/attribution graph, read examples from your dataset, try logit lens or steering, etc.
- Set a 5-minute timer and brainstorm things you're curious about or directions to try.
- If you're confused/curious about something, set a 5 minute timer and brainstorm what could be happening.

Other advice:

- Before any >30 minute experiment, stop and brainstorm alternatives. Is this *really* the fastest way to gain information?
- It's totally fine to pause for half a day to go learn some key background knowledge.
- Get in the habit of keeping a research log of your findings and a "highlights" doc for the really cool stuff.
    - If applicable, it can be cool to have your research log be a slack/discord channel
- Remember: when exploring and thinking through how to explain mysterious phenomena, most of your probability mass should be on "something I haven't thought of yet."
- Practice following your curiosity, but be aware that it'll often lead you astray at first. When it does, pay attention! What can you learn from this?

Practicing Understanding

If exploration goes well, you'll start to form hunches about the problem. E.g. thinking that you are successfully (linearly) probing for some concept. Or that you found a direction that mediates refusal. Or that days of the week are represented as a circle in a 2D subspace.

Once you have this, you want to go to figure out if it's actually true. Be warned, the feeling of "being really convinced that it's true" is very different from actually being true. Part of being a good researcher is being good enough at testing and falsifying your pet hypotheses that, when you fail to falsify one, there's a good chance that it's true. But you're probably not there yet.

Note: While I find it helpful to think of these as discrete stages, often you'll be flitting back and forth. A great way to explore is coming up with guesses and micro-hypotheses about what's going on, running a quick experiment to test them, and integrating the results into your understanding of the problem, going back to the drawing board.

Your North Star: convince yourself a hypothesis is true or false. The key mindset is skepticism. Advice:

- Before testing a hypothesis, set a five-minute timer and brainstorm, "What are the ways this could be false?"
- Alternatively, write out the best possible case for your hypothesis and see where the argument feels weak.

    - Try using an LLM with an anti-sycophancy prompt ("My friend wrote this and wants brutal feedback...") to red-team your arguments - it probably won't work, but might be helpful
- Or set a 5 minute timer and brainstorm alternative explanations for your observations

You then want to convert these flaws and alternative hypotheses into concrete experiments. **Experiment design is a deep skill**. Honestly, I'm not sure how to teach it other than through experience. But one recommendation is to pay close attention to the experiments in papers you admire and analyze what made them so clever and effective. I also recommend that, every time you feel like you've (approximately) proven or falsified a hypothesis, adding them to a running doc of "things I believe to be true" with hypotheses, experiments, and results.

Using LLMs for Research Code

In my opinion, coding is one of the domains where LLMs are most obviously useful. It was very striking to me how much better my MATS scholars were six months ago than 12 months ago, and I think a good chunk of this is attributable by them having much better LLMs to use. If you are not using LLMs as a core part of your coding workflow, I think you're making a mistake.

- **Use Cursor:** It's VS Code with fantastic AI integration. Make sure to add the docs for libraries with @ so the AI has context. The $20/month plan is worth it, if possible, and there's a free student version.
    - Claude Code is tempting but bad for learning and iteration. I'd use it for throwaway things and first drafts - if the draft has a bunch of bugs, go read the code yourself/throw it away and start again. Cursor facilitates reading the AI's code better than Claude code does IMO
- **A caveat:** If learning a new library (like in ARENA), first try writing things yourself. Use the LLM when stuck, not to replace the learning process.
- Later on, when thinking about writing up results, if key experiments were mostly vibe-coded, I recommend re-implementing them by hand to make sure no dumb LLM bugs slipped in.

Interlude: What's New In Mechanistic Interpretability?

Feel free to skip to the "what should I do next°" part

Things move fast in mechanistic interpretability. Newcomers to the field who've kept up from afar are often pretty out of date. Here's what I think you need to know, again, filtered through my own opinions and biases.

Avoiding Fads

This interlude is particularly important because **the field often has fads**: lines of research that are very popular for a year or so, make some progress and find many limitations, and then the field moves on. But if you're new, and catching up on the literature, you might not realise. I often see people new to the field working on older things, that I don't think are too productive to work on any more. Historical fads include:

- Interpreting toy models trained on algorithmic tasks (e.g. my grokking work)
  - I no longer recommend working on this, as I think we basically know that "sometimes models trained on algorithmic tasks are interpretable", and they're sufficiently artificial and divorced from real models that I am pessimistic about deeper and more specific insights generalising
- Circuit analysis via causal interventions on model components (e.g. the IOI paper)
  - This is slightly more complicated. I think that's worth learning about, and techniques like activation and attribution patching are genuinely useful.
  - But the core problem is that once you got a sparse subgraph of a model responsible for a task, there wasn't really a "what next?". This didn't tend to result in deeper insight because the nodes (eg layers or maybe attention heads) weren't monosemantic, and it was often more complicated than naive stories suggested but we didn't have the tools to dig deeper.
  - It was pretty cool to see that this was possible at all, but there have been more than enough works in this area that the bar for a novel contribution is now much higher.
  - Simply identifying a circuit is no longer enough; you need to use that circuit to reveal a deeper, non-obvious property of the model. I recommend exploring attribution-graph style approaches°
- We're at the tail end of a fad of incremental sparse autoencoder research[15] (i.e. focusing on simple uses and refinements of the basic technique)
  - Calling this one a fad is probably more controversial (if only because it's more recent).

- The *specific* thing I am critiquing is the spate of papers, including ones I was involved in, that are about incremental improvements to the sparse autoencoder architecture, or initial demonstrations that you can apply SAEs to do things, or picking some downstream task and seeing what SAEs do on it.
    - I think this made some sense when it seemed like SAEs could be a total gamechanger for the field, and where we were learning things from each new such paper. I think this moment has passed; I do not think they were a gamechanger in the way that I hoped they might be. See more of my thoughts here°.
- I am *not* discouraging work on the following:
    - Attribution graph-based circuit analysis, which I don't think has played out yet - see a recent overview of that sub-field I co-wrote°.
    - Trying meaningfully different approaches to dictionary learning (eg SPD or ITDA), or things targeted to fix conceptual limitations of current techniques (eg Matryoshka).
    - Using SAEs as a tool, whether as part of a broader project investigating weird phenomena in model biology, or as a baseline/approach on some downstream task. The key is that the project's motivation should *not* just be "what if we used SAEs for X?" unless there's a good argument
        - I particularly recommend them for tasks where you don't know exactly what you're looking for, e.g. trying to explore some mysterious phenomena

Note that I am putting this after stage 2 because I think that **for initial throwaway projects you should *not* be stressing about novelty and avoiding fads** - your goal is just to learn. But as we move into stage 3 you should start to be a bit more mindful about choosing more exciting/impactful projects where possible.

Also, take these as nudges and recommendations, not as instructions. If there's a direction you believe in that fits the things I'm critiquing, maybe I'm just wrong, maybe your thing is an exception, go wild, see what happens.

What's New In Mech Interp?

OK, so those are my hot takes on what not to do. What should you do? I think that some really cool new opportunities have opened up in mech interp over the last year, and newcomers may not have come across these. Here are some of the key themes in my favorite papers over the last year, that I'd love to see readers build on:

1. **Downstream tasks**. If our North Star is to be able to do useful things with interpretability, a great way to test our techniques is on real-world tasks, to see if they beat baselines or enable us to do a new thing
    a. Even if we can't directly test on the tasks we most care about, if models can let us do new things in *some* domains, that's reasonable evidence that we're learning something real in a way that is hard to fake.
    b. One of my favourite examples is **auditing games**: Marks et al. is a fantastic paper that trained a model to have a hidden goal and then had different teams compete to find this goal, some with interpretability techniques, some without. Hypothesis: If interpretability does something real, then interp teams should win.
        i. This study itself was a bit ambiguous (the mech interp teams were 1st and 3rd, a black box team was 2nd), but I think this is a fantastic way to test interpretability techniques, and has a ton of potential for follow-on work evaluating interpretability in different kinds of games.
        ii. The general idea is just that there is a model with some known hidden property and different techniques are used to see which can elicit it. Things like having different teams, allowed to use different techniques, etc. is just one implementation.
2. **Model organisms**: The auditing games paper was made possible by the fact that they were able to make a model with a hidden goal[16], a model organism to study. In general, we're collecting techniques like synthetic document fine-tuning to make really interesting model organisms.
    a. This kind of thing has a lot of potential! If we want to make a lie detector, a core challenge is that we don't know how to test if it works or not. But if we can insert beliefs or deceptive behaviours into a model, many more projects become possible
    b. A great intro project is playing around with open source model organisms, e.g. from Cywinski et al
3. **Practice on the real AGI Safety problems**: Historically, interpretability could only practice on very dull toy problems like modular addition. But we now have models that

exhibit complex behaviors that seem genuinely relevant to safety concerns, and we can just study them directly, making it far easier to make real progress.

   a. E.g. Rajamanoharan et al° debunking assumed self-preservation, and Goldowsky-Dill et al probing for deception

   b. Weird behaviours: models can insider trade then lie about it, tell when they're being evaluated (and act differently), fake alignment, reward hack, and more.

4. **Real-World Uses of Interpretability**: Model interpretability-based techniques are starting to have genuine uses in frontier language models!
   a. Linear probes, one of the simplest possible techniques, are a highly competitive way to cheaply monitor systems for things like users trying to make bioweapons.

   b. I find it incredibly cool that interpretability can actually be useful, and kind of embarrassing that only a decade-old technique seems very helpful. Someone should do something about that. Maybe that someone could be you!

   c. This needs a very different kind of research: careful evaluation, comparison to strong baselines, and refinement of methods

5. **Attribution graph-based circuit analysis**. The core problem with trying to analyze circuits in terms of things like a model's attention heads and layers is that often these things don't actually have a clear meaning. Attribution graphs use techniques like transcoders, popularized in Anthropic's model biology work, to approximate models with a computational graph with meaningful nodes.
   a. See this cross-org blog post° for the ongoing follow-on work across the community, and an open problems list I co-wrote![17]

   b. You can make and analyse your own attribution graphs on Neuronpedia°

6. **Understanding model failures**: Models often do weird things. If we were any good at interpretability, we should be able to understand these. Recently, we've seen signs of life!
   a. Meng et al on why some models think 9.8 < 9.11

   b. A line of work studying emergent misalignment - why training models on narrowly evil tasks like writing insecure code turns them into Nazis - has found some insights. Wang et al found this was driven by sparse autoencoder latents[18] associated with movie villains, and in Turner et al° we found that the model *could* have learned the narrow solution, but this was in some sense less "efficient" and "stable"

7. **Automated interpretability**: Using LLMs to automate interpretability. We saw signs of life on this from Bills et al and Shaham et al, but LLMs are actually good now! It's now

possible to make basic interpretability agents that can do things like solve auditing games[19]. And interpretability agents are the worst they'll ever be[20].

8. **Reasoning model interpretability**: All current frontier models are reasoning models —models that are trained with reinforcement learning to think[21] for a while before producing an answer. In my opinion, this requires a major rethinking of many existing interpretability approaches[22], and calls for exploring new paradigms. IMO this is currently being neglected by the field, but will become a big deal.

    a. In Bogdan et al, we explored what a possible paradigm could look like. Notably, there are far more interesting and sophisticated black box techniques with reasoning models, like resampling the second half of the chain of thought, or every time the model says a specific kind of sentence, deleting and regenerating that sentence.

A Pragmatic Vision for Mech Interp

Attentive readers may notice that the list above focuses on work to do with understanding the more qualitative high-level properties of models, and not ambitious reverse engineering. This is largely because, in my opinion, the former has gone great, while we have not seen much progress towards the fundamental blockers on the latter.

I used to be very excited about ambitious reverse engineering, but I currently think that the dream of completely reverse engineering a model down to something human understandable seems basically doomed. My interpretation of the research so far is that models have some human understandable high-level structure that drives important actions, and a very long tail of increasingly niche and irrelevant heuristics and biases. For pragmatic purposes, these can be largely ignored, but not if we want things like guarantees, or to claim that we have understood most of a model. I think that trying to understand as much as we can is still a reasonable proxy for getting to the point of being pragmatically useful, but think it's historically been too great a focus of the field, and many other approaches seem more promising if our ultimate goals are pragmatic.

In some ways, this has actually made me more optimistic about interpretability ultimately being useful for AGI safety! Ambitious reverse engineering would be awesome but was always a long shot. But I think we've seen some real results for pragmatic approaches to mechanistic interpretability, and feel fairly confident we are going to be able to do genuinely useful things that are hard to achieve with other methods.

Stage 3: Working Up To Full Research Projects

Once you have a few mini-projects done, you should start being more ambitious. You want to think about gaining the deeper (medium/slow) skills, and exploring ideation and distillation.

However, you should still expect projects to often fail, and want to lean into breadth over depth and avoid getting bogged down in an unsuccessful project you can't bear to give up on. To resolve this tension, I recommend **working in 1-2 week sprints**. At the end of each sprint, reflect and make a deliberate decision: **continue, or pivot?** The default should be to pivot unless the project feels truly promising. It's great to give up on things, if it means you spend your time even better! But if it's going great, by all means continue.

This strategy should mean that you eventually end up working on something longer-term when you find something *good*, but don't just get bogged down in the first ambitious idea you tried.

I recommend reviewing the list of skills earlier and just for each one, reflecting for a bit on how on top of it you think you feel and how you could intentionally practice it in your next project. Then after each sprint, before deciding whether to pivot, take an hour or two to do a post-mortem: what did you learn, what progress did you make on different skills, and what would you do differently next time? Your goal is to learn, and you learn much better if you make time to actually process your accumulated data!

Key Research Mindsets

One way to decompose your learning is to think about research mindsets: the traits and mindsets a good researcher needs to have, that cut across many of these stages. See my blog post on the topic for more°, but here's a brief view of how I'm currently thinking about it.

1. **Skepticism/Truth-seeking:** The default state of the world is that your research is false, because doing research is hard. Your north star should always be to find *true* insights[23]
    a. It generally doesn't come naturally to people to constantly aggressively think about all the ways their work could be false and make a good faith effort to test it. You can learn to do better than this, but it often takes practice.

   b. This is crucial in understanding, somewhat important in exploration, and crucial in distillation.
   c. A common mistake is to grasp at straws to find a "positive" result, thinking that nothing else is worth sharing.
      i. In my opinion, negative or inconclusive results that are well-analyzed are much better than a poorly supported positive result. I'll often think well of someone willing to release nuanced negative results, and poorly of someone who pretends their results are better than they are.
2. **Prioritization:** Your time is scarce. Research involves making a bunch of decisions that are essentially searching through a high-dimensional space. The difference between a great and a mediocre researcher is being able to make these decisions well.
   a. If you have a good mentor, you can lean on them for this at first, but you will need to learn how to do this yourself eventually.
   b. This is absolutely crucial in exploration and ideation, but fairly important throughout.
   c. A good way to learn this one is to reflect on decisions you've made after the fact, eg in a sprint post-mortem, and think about how you could have made them better, and what generalisable lessons to take to the future
3. **Productivity**[24]**:** The best researchers I've worked with get more than twice as much done as the merely good ones. Part of this is good research taste and making good prioritization decisions, but part of this is just being good at getting shit done.
   a. Now, this doesn't necessarily mean pushing yourself until the point of burnout by working really long hours. Or cutting corners and being sloppy. This is about productivity integrated over the long term.
      i. For example, sometimes the most productive thing to do is to hold off on starting work, set a 5 minute timer, brainstorm possible things to do next, and then pick the best idea
   b. This takes many forms, and the highest priority for you:
      i. Know when to write good code without bugs, to avoid wasting time debugging later, and when to write a hacky thing that just works.
      ii. Know the right keyboard shortcuts to move fast when coding.

iii. Know when to ask for help and have people who can help you get unblocked where appropriate.

iv. Be good at managing your time and tasks so that once you've decided what the highest priority thing to work on is, you in fact go and work on it.

v. Be able to make time to achieve deep focus on the key problems.

c. Exercise: Occasionally **audit your time**. Use a tool like Toggl for a day or two to log what you're doing, then reflect: where did time go? What was inefficient? How could I do this 10% faster next time?

i. The goal isn't to feel guilty, but to spot opportunities for improvement, like making a utility function for a tedious task.

4. **Knowing the literature**: At this point, there's a lot of accumulated wisdom (and a lot of BS) in prior papers, in mech interp and beyond.

a. This cuts across all stages:

i. In ideation, you don't want to accidentally reinvent the wheel. And often great ideas are inspired by prior work

ii. In exploration, you want to be able to spot connections, borrow interesting techniques, etc

iii. In understanding, you want to know the right standards of proof to check for, the best techniques to use, alternative hypotheses (that may have been raised in other works), etc

iv. In distillation, when writing a paper you're expected to be able to contextualise it relative to existing work (i.e. write a related work section[25]) which is important for other researchers knowing whether to care. And if you don't know the standard methods of proof, key baselines everyone will ask about, key gotchas to check for etc, no one will believe your work.

b. LLMs are an incredibly useful tool here. GPT-5 thinking or Claude 4 with web search are both pretty useful tools here, as are the slower but more comprehensive deep research tools (Note that Google's is available for free, as of the time of writing)

i. I recommend using these regularly and creatively throughout a project.

ii. You don't necessarily need to go and read the works that get surfaced, but even just having LLM summaries can get you more awareness of what's out there, and over time you'll build this into deeper knowledge.

c. Of course, when there *does* seem to be a very relevant paper to your work, you should go do a deep dive and read it properly, not just relying on LLM summaries.

d. Don't stress - deep knowledge of the literature takes time to build. But you want to ensure you're on an upwards gradient here, rather than assuming the broader literature is useless

e. On the flip side, many papers *are* highly misleading/outright false, so please don't just critically believe them![26]

Okay, so how does this all tie back to the stages of research? Now you're going to be thinking about all four. We'll start by talking about how to deepen your existing skills with exploration and understanding, and then we'll talk about what practicing ideation and actually writing up your work should look like.

Deepening Your Skills

You'll still be exploring and understanding, but with a greater focus on rigor and the slower skills. In addition to the thoughts when discussing mindsets above, here's some more specific advice

- **Deeper Exploration** is about internalizing the mindset of maximising productivity, which here means maximising information gain per unit time. Always ask, "Am I learning something?"
  - *Avoid Rabbit Holes:* A common mistake is finding one random anomaly and zooming in on it. Knowing when to pivot is crucial. Set a timer every hour or two to zoom out and ask if you're making progress.
    - I recommend any time you notice yourself feeling a bit stuck or distracted or off track, setting a five minute timer and thinking about what could I be doing next, what should I be doing next, and am I doing the most important thing?
  - *Avoid Spreading Yourself Too Thin:* Doing lots of things superficially means none of them will be interesting.
  - If you have spent more than five hours without learning something new, you should probably try a different approach
    - And if you have spent more than two days without learning something new, you should seriously consider pivoting and doing something else.
  - To practice prioritization, be intentional about your decisions: write down *why* you think an experiment is the right call, and later reflect on whether you

were right. This makes your intuitions explicit and easier to update.

- **Deeper Understanding** is about practicing skepticism and building a bulletproof case. Red-team your results relentlessly.
  - Some experiments are much more impactful and informative than others! Don't just do the first experiment that pops into your head. Think about the key ways the hypothesis *could* be false, and how you could test that. Or about whether a skeptic could explain away a positive experimental results
    - A useful exercise is imagining you're talking to a really obnoxious skeptic who keeps complaining that they don't believe you and coming up with arguments for why your thing is wrong. What could you do such that they don't have a leg to stand on?
  - Of course, there's also an element of prioritization. Sometimes a shallow case that could be wrong is the right thing to aim for, if you're working on an unimportant side claim/something that seems super plausible on priors, at which point you should just move on and do something else more interesting.
  - Exercise: To practice spotting subtle illusions, try red-teaming papers you read, thinking about potential flaws, and ideally run the experiments yourself.

Doing Good Science

- **Avoid cherry-picking**: Researchers can, accidentally or purposefully, produce evidence that looks more compelling than it actually is. One classic way is cherry-picking: presenting only the examples that look most compelling.
  - When you write up work, always include some randomly selected examples, especially if you present extensive qualitative analysis of specific things. It's fine to put this in the appendix if space is scarce, but it should be there.
- **Use baselines**: A common mistake is for people to try to show a technique works by demonstrating it gets 'decent' results, rather than showing it achieves better results than plausible alternatives that people might have used or are standard in the field. If you want people to e.g. use your cool steering vector results you need to show it beats changing the system prompt.
- **Don't sandbag your baselines**: Similarly, it's easy to put in much more effort finding good hyperparameters for your technique than for your baselines. Try to make sure you're achieving comparable results with your baselines that prior work in the field has.
- **Do ablations on your fancy method**: It's easy for people to have a fancy method with lots of moving parts, when many actually are unnecessary. You should always try

removing one part and see if the method breaks. Do this for each part.
    - For example, the original unlearning method in the RMU paper claimed it was based on finding a meaningful steering vector, until follow-up work found that it was just about adding a vector with really high norm that broke the model, and a random vector performed just as well.
- **(Informally) pre-register claims**: It's important to clearly track which experimental results were obtained before versus after you formulated your claim. Post-hoc analysis (interpreting results after they're seen) is inherently less impressive than predictions confirmed by pre-specified experiments
- **Be reproducible**: Where practical, share your code, data and models.
    - If you have time, make sure that it runs on a fresh machine and include a helpful readme that links to key model weights and datasets.
    - This both means others can check if your work is true and makes it more likely people will believe and build on your work[27] because they can see replications that are more likely to exist and because it's now low friction.
- **Simplicity:** Bias towards trying the simple, obvious methods first. Fancy techniques can be a trap. Good research is pragmatic, not about showing off.
    - If you're designing a fancy technique/experiment, each new detail is one more thing that can break
    - If trying to explain something mysterious, novice researchers often neglect simple, dumb hypotheses like "maybe MLP0 is incredibly important on *every* input, and there's nothing special going on with my prompt"
- **Be qualitative *and* quantitative**: One of the major drivers of progress of modern machine learning is being quantitative, having benchmarks and showing that a technique increases numbers on them. One of the key drivers of progress in mech interp is an openness to qualitative research: summary statistics lose a ton of information. What can we learn by actually looking deeply into what's happening?
    - In my opinion, the best research tries to get the best of both worlds. It tries to understand what's happening via qualitative analysis and then validates it with more quantitative methods. If your paper only does one, it's probably missing out
- **Read your data**: A fantastic use of time, especially during the exploration phase, is just actually reading the data you're working with, or model chains of thought and responses.
    - Often, the quality of the data is a crucial driver of the results of your experiments. Often, it is quite bad.

    - Sometimes most of the work of a project is in noticing flaws in your data and making a better data set. Time figuring this out is extremely well spent.
    - Ditto, include random examples of the data in an appendix for readers to do spot checks of their own.
- **Don't reinvent the wheel**: A common mistake in mech interp is doing something that's already been done[28]. We have LLM-powered literature reviews now. You have way less of an excuse. Check first!
- **Excitement is evidence of bullshit**: Generally, most true results are not exciting, but a fair amount of false results are. So from a Bayesian perspective, if a result is exciting and cool, it's even more likely to be false than normal!
    - Resist the impulse to get really excited! The correct attitude to exciting results is deep skepticism until you have tried really hard to falsify it and run out of ideas.
- **Get the stats right**: There's a lot of traps to avoid when it comes to things like having the right sample size, knowing what you can and cannot infer from the evidence you have, etc. One of my MATS alumni, Paul Bogdan, has a great blog post on this here°.

Practicing Ideation

Okay, so you want to actually come up with good research ideas to work on. What does this look like? I recommend breaking this down into **generating ideas** and then **evaluating** them to find the best ones.

To generate ideas, I'd often start with just taking a blank doc, blocking out at least an hour, and then just writing down as many ideas as you can come up with. Aim for quantity over quality. Go for at least 20.

There are other things you can do to help with generation:

- Throughout your previous sprints, every time you had an idle curiosity or noticed something weird, write it down in one massive long-running doc.
- Likewise, when reading papers, note down confusions, curiosities, obviousnesses to do.

Okay, so now you have a big list. What does finding the best ones look like?

- Ideally, if you have a mentor or at least collaborators, you can just ask them to rate them.
    - If you do this, rate them yourself privately out of 10 before you look at their responses. Compare them and every time you have substantially different

numbers, talk to the mentor and try to figure out why your intuitions disagree. This is a great source of supervised data for research taste.

- Even if you don't have a mentor, I think that just going through, rating each idea yourself based on gut feel and sorting is as good a way to prune down a long list as any
- For the top few, I recommend trying to answer a few questions about them.
  - What would success look like here?
  - How surprised would I be if I did this for a month and nothing interesting had happened?
  - What skills does this require? Do I have them/could I easily gain them?
  - What models, data, computational resources, etc. does this require?
  - How does this compare to what the most relevant prior work did? Can I check for prior work and see if anything relevant comes up?

Research Taste Exercises

Gaining research taste is slow because the feedback loops are long. You can accelerate it with exercises that give you faster, proxy feedback. (Credit to Chris Olah for inspiration here)

- If you have a mentor, query their taste for fast data and try to imitate it. Concretely:
  - Before each meeting, write a list of questions, then try to write up predictions for what the mentor will say, then actually ask the mentor, see what happens, and compare. If there are discrepancies, chat to the mentor and try to understand why.
  - Likewise, if the mentor makes a suggestion or asks a question you didn't expect, try to ask questions about where the thought came from.
  - Regularly paraphrase back to the mentor in your own words what you think they're saying, and then ask them to correct anything you're wrong about[29]
- **Learning from papers as "offline data":** When you read a paper, don't just passively consume it. Read the introduction, then stop. Try to predict what methods they used and what their key results will be. Then, continue reading and see how your predictions compare. Analyze why the authors made different choices. This trains your intuition on a much larger and faster dataset than your own research.

It's also worth dwelling on what research taste actually is. See my post° for more, but I break it down as follows:

1. **Intuition (System 1):** This is the fast, gut-level feeling - what people normally think of when they say research taste. A sense of curiosity, excitement, boredom, or skepticism about a direction, experiment, or result.
2. **Conceptual Framework (System 2)**: This is deep domain knowledge and understanding of underlying principles.
3. **Strategic Big Picture**: Understanding the broader context of the field. What problems are important? What are the major open questions? What approaches have been tried? What constitutes a novel contribution?

Write up your work!

At this stage, you should be thinking seriously about how to write up your work. Often, writing up work is the first time you really understand what a project has been about, or you identify key limitations, or experiments you forgot to do. You should check out my blog post on writing ML papers° for much more detailed thoughts (which also apply to high-effort blog posts!) but I'll try to summarize them below.

Why aim for public output?

If producing something public is intimidating, for now, you can start by just writing up a private Google Doc and maybe share it with some friends or collaborators. But I heavily encourage people to aim for public output where they can. Generally, your research will not matter if no one reads it. The goal of research is to contribute to the sum of human[30] knowledge. And if no one understands what you did, then it doesn't matter.

Further, if you want to pursue a career in the space, whether a job, a PhD, or just informally working with mentors, **public research output is your best credential**. It's very clear and concrete proof that you are competent, can execute on research and do interesting things, and this is exactly the kind of evidence people care about seeing if they're trying to figure out whether they should work with you, pay attention to what you're saying, etc. It doesn't matter if you wrote it in a prestigious PhD program or as a random independent researcher, if it's good enough then people care.

There are a few options for what this can look like:

- A blog post (e.g. on a personal blog or LessWrong) - the simplest and least formal kind
- An Arxiv paper - much more legible than a blog post, and honestly not much extra effort if you have a high-quality blog post[31]

- A workshop paper[32] (i.e. something you submit for peer review to a workshop, typically part of a major ML conference, the bar is much lower than for a conference paper)
- A conference paper (the equivalent of top journals in ML, there's a reasonably high quality bar[33], but also a *lot* of noise[34])

If this all seems overwhelming, starting out with blog posts is fine, but I think people generally overestimate the bar for arxiv or workshop papers - if you think you learned something cool in a project, this is totally worth turning into a paper!

How to write stuff up?

The core of a paper is the narrative. Readers will not take away more than a few sentences worth of content. Your job is to make sure these are the right handful of sentences and make sure the reader is convinced of them.

You want to distill your paper down into one to three key claims (your contribution), the evidence you provide that the contribution is true, the motivation for why a reader should care about them, and work all of this into a coherent narrative.

Iterate: I'm a big fan of writing things iteratively. You first figure out the contribution and narrative. You then write a condensed summary, the abstract (in a blog post, this should be a TL;DR/executive summary - also very important!). You then write a bullet point outline of the paper: what points you want to cover, what evidence you want to provide, how you intend to build up to that evidence, how you want to structure and order things, etc. If you have mentors or collaborators, the bullet point outline is often the best time to get feedback. Or the narrative formation stage, if you have an engaged mentor. Then write the introduction, and make sure you're happy with that. Then (or even before the intro) make the figures - figures are incredibly important! Then flesh it out into prose. People spend a *lot* more time reading the abstract and the intro than the main body, especially when you account for all the people who read the abstract and then stop. So you should spend a lot more time per unit word on those.

LLMs: I think LLMs are a really helpful writing tool. They're super useful for getting feedback, especially if writing in an unfamiliar style like an academic ML paper may be for you. Remember to use anti-sycophanty prompts so you get real feedback. However, it's often quite easy to tell when you're reading LLM written slop. So use them as a tool, but don't just have them write the damn thing for you. But if you e.g. have writer's block, having an LLM help you brainstorm or produce a first draft for inspiration, and can be very helpful.

Common mistakes

- **The reader does not have context**: Your paper will be clear in your head, because you have just spent weeks to months steeped in this research project. The reader has not. You will overestimate how clear things are to the reader, and so you should be massively erring in the other direction and spelling everything out as blatantly as possible.
    - **This is an incredibly common mistake** - assume it will happen to you
    - The main solution is to get feedback from people with enough research context that they can actually engage and who are also willing to give you substantial negative feedback.
        - Notice the feeling of surprise when people are confused by something you thought was clear. Try to understand why they were confused and iterate on fixing it until it's clear.
- **Writing is not an afterthought**: People often do not prioritize writing. They treat it like an annoying afterthought and do all the fun bits like running experiments, and leave it to the last minute.
- **Acknowledge limitations**: There is a common mistake of trying to make your work sound maximally exciting. Generally, the people whose opinions you most care about are competent researchers who can see through this kind of thing
- **Good writing is simple**: There's a tendency towards verbosity or trying to make things sound more complex and fancy than they actually are, so they feel impressive. I think this is a highly ineffective strategy
- **Remember to motivate things**: It will typically not be obvious to the reader why your paper matters or is interesting. They do not have the context you do. It is your job to convince them, ideally in the abstract or perhaps intro, why they should care about your work, lest they just give up and stop reading.

Mentorship, Collaboration and Sharing Your Work

A common theme in the above is that it's incredibly useful to have a mentor, or at least collaborators. Here I'll try to unpack that and give advice about how to go about finding one.

Though it's also worth saying that many mentors are not actually great researchers and may have bad research taste or research taste that's not very well suited to mech interp. What you do about this is kind of up to you.

So what does a research mentor actually do?

A good mentor is an incredible accelerator. Dysfunctional as academia is, there is a reason it works under the apprenticeship-like system of PhD students and supervisors. When I started supervising, I was very surprised at how much of a difference a weekly check in could make! Here's my best attempt to breakdown how a good mentor can add value:

- **Suggest research ideas** when you're starting out, letting you bypass the hardest skill (ideation) to focus on execution.
- **Help you prioritize** which experiments to run, lending you their more experienced judgment, so you get more done.
- **When to pivot**: if your research direction isn't working out, having a mentor to pressure you to pivot can be extremely valuable[35]
- **Provide supervised data for research taste**: For the slow/very-slow skills like coming up with research ideas, and prioritization, a *far* faster way to gain them at first is by learning to mimic your mentor's.
- **Act as an interface to the literature**: pointing you to the relevant work before you've built up deep knowledge yourself. Flagging standard baselines, standard metrics, relevant techniques, prior work so you don't reinvent the wheel, etc.
- **Red-team your results**, helping you spot subtle interpretability illusions and flaws in your reasoning that you're too close to see.
- **Point out skills you're missing** that you didn't even notice were skills. Generally guide your learning and help you prioritize
- **Walk you through communicating your work**, helping you distill your findings and present them clearly to the world.
- **Motivation/accountability**: Many find it extremely helpful to have someone, even if very hands-off, who they present work to, so they feel motivated and accountable (especially if they e.g. want to impress the mentor, want a job, etc. Of course, these also increase stress!)
    - To those prone to analysis paralysis, being able to defer to a mentor on uncertain decisions can be highly valuable
- **References**: Having a mentor who can vouch for your skill is very helpful, especially if they know people who may be hiring you in future.

Advice on finding a mentor

Here are some suggested ways to get some mentorship while transitioning into the field. I discuss higher commitment ways, like doing a PhD or getting a research job, below.

Note: whatever you do to find a mentor, having evidence that you can do research yourself, that is, public output that demonstrates ability to self-motivate and put in effort, and ideally demonstrates actually interesting research findings, is incredibly helpful and should be a priority.

Mentoring programs

I think mentoring programs like MATS are an incredibly useful way into the field, you typically do a full-time, several month program where you write a paper, with weekly check-ins with a more experienced researcher. Your experience will vary wildly depending on mentor quality, but at least for my MATS scholars, often people totally new to mech interp can publish a top conference paper in a few months. See my MATS application doc for a bunch more details.

There's **a wide range of backgrounds** among people who do them and get value - people totally new to a field, people with 1+ years of interpretability research experience who want to work with a more experienced mentor, young undergrads, mid-career professionals (including a handful of professors), and more.

My MATS 10.0 applications are open, due **Dec 23 2025**

Other programs (which I think are generally lower quality than MATS, but often still worth applying to depending on the mentor)

- *Full-time/In-person:* MATS, Pivotal, LASR, PIBBSS
- *Part-time/Remote:* SPAR, MARS

Sending A Good Cold Email

You can also take matters into your own hands and try to convince someone to be your mentor. Reaching out to people, ideally via a warm introduction, but even just via a cold email, can be highly effective. However, I get lots of cold emails and I think many are not very effective, so here's some advice:

- **Don't just email the most prominent people**. A lot of people will just email the most prominent people in the field and ask for mentorship. This is a bad plan! These people are very busy and they also get lots of emails. I just reflexively respond to any email requesting mentorship with "please apply to my MATS cohort".

    - However, there are lots of less prominent people who can provide a bunch of useful mentorship. These people are much more likely to be excited to get a cold email, to have time to engage, potentially even the spare capacity to properly mentor a project.
    - I think that many people who've recently joined my team or people who worked on a great paper with me during MATS are able to add a lot of value to people new to the field. And I would recommend reaching out to them!
        - For example, Josh Engels, a new starter on my team, said he would happily receive more cold emails (as of early Sept 2025).
        - As a general heuristic, email first authors of papers, not fancy last authors.
- **Start small**: Don't email someone you've never interacted with before asking if they want to kind of officially mentor you on some project. That's a big commitment.
    - It's much better to be like, I'd be interested in having a chat about your paper or my work building on your paper.
    - Or just asking if they're down to have a chat giving you feedback on some project ideas, etc.
    - And if this goes well, it may organically turn into a more long-term mentoring relationship!
- **Proof of work**: Demonstrate that you are actually interested in this person specifically, not just spamming tons of people.
    - Show that you've engaged with their work, say something intelligent about it, have some questions.
        - In the era of LLMs, this is less of a costly signal that you've actually taken an interest in this person specifically than it used to be, admittedly
        - But linking to some research you did building on their work I think is still reasonably costly, and very flattering to people.
- **Prioritize aggressively**. Assume the reader will stop reading at any moment, so put your most critical and impressive information first.
- **Explain who you are**: If you're emailing someone who gets more emails than they have capacity to respond to, they're going to be prioritizing. A key input into this is just who you are, what have you done, have you done something interesting that shows promise, do you have relevant credentials, etc. I personally find it very helpful if people just say the most impressive things about them in the first sentence or two.
    - To do this without seeming arrogant, you could try: "I'm sure you must get many of these emails. So to help you prioritise, here's some key info about me"

- Use **bolding** for key phrases to make your email easily skimmable.
- **Be concise**. One thing I would often appreciate is a short blurb summarizing your request with a link to a longer document for details if I'm interested.
- **Quick requests**: Generally, my flow when reading emails is that I will either immediately respond or never look at it again. I'm a lot more likely to immediately respond if I can do so quickly. If you do want to email a busy person, have a clear, concrete question up front that they might be able to help with.

Community & collaborators

Much easier than finding a mentor is finding collaborators, other people to work on the same project with, or just other people also trying to learn more about mech interp, who you can chat with and give each other feedback:

- **In-Person:** Local AI Safety hubs (London, Bay Area, etc.), University groups, ML conferences (e.g., the NeurIPS Mech Interp workshop I co-organize), EAG/EAGx conferences.
  - If you're a student, see if there's a lab at your university that has some people interested in interpretability. There may be interested PhD students even if no professor works on it
- **Online**: These are also good places to meet people! I recommend sharing work for feedback, or just asking about who's interested in what you're interested in, and trying to DM the people who engage/seem interested, and seeing what happens
  - Open Source Mechanistic Interpretability Slack
  - Eleuther Discord (interpretability-general)
  - Mech Interp Discord

Staying up to date: Another common question is how to stay up to date with the field. Honestly, I think that people new to the field should not worry that much about this. Most new papers are irrelevant, including the ones that there is hype around. But it's good to stay a little bit in the loop. Note that the community has substantial parts both in academia and outside, which are often best kept up with in different ways.

- LessWrong and the AlignmentForum are a reasonable place to keep up to date with the less academic half
- Twitter is a confusing, chaotic place that is an okay place to keep up with both. It's a bit unclear who the right people to follow.

- Chris Olah doesn't tweet much, but it's high quality when he does.
- I will tweet about all of my interpretability work and sometimes others.

Careers

Where to apply

- Anthropic's interpretability team roles: research scientist, research engineer, research manager
- OpenAI's interpretability team roles
- My team at Google DeepMind will hopefully be hiring in early 2026! Watch this space
- Transluce -- a nonprofit research lab
- Goodfire -- a mech interp startup that are hiring a bunch.
  - They recently raised a $50 million Series A and as of the time of writing are trying to both have people focused on products, and people focused on more fundamental research
- The UK government's AI Security Institute's interpretability team (not currently hiring)

Applying for grants

For people trying to get into mech interp via the safety community, there are some funders around open to giving career transition grants to people trying to upskill in a new field like mech interp. Probably the best place I know of is Open Philanthropy's Early Career Funding.

Explore Other AI Safety Areas

Mech interp isn't the only game in town! There's other important areas of safety like Evals, AI Control, and Scalable Oversight, the latter two in particular seem neglected compared to mech interp. The GDM AGI Safety Approach gives an overview of different parts of the field. If you're doing this for safety reasons, I'd check if there's other, more neglected subfields, that also appeal to you!

What do hiring managers look for

Leaving aside things that apply to basically all roles, like whether this person has a good personality fit (which often just means looking out for red flags), here's my sense of what hiring managers in interpretability are often looking for.

A useful mental model is that from a hiring manager's perspective, they're making an uncertain bet with little information in a somewhat adversarial environment. Each applicant wants to present themselves as the perfect fit. This means managers need to rely on signals that are hard to fake. But it's quite difficult to get that much info on a person before you actually go and work with them a bunch.

Your goal as a candidate is to provide compelling, hard-to-fake evidence of your skills. The best way to do that is to simply do good research and share it publicly. If your research track record is good enough, interviews may just act as a check for red flags and to verify that you can actually write code and run experiments well.

Key skills:

- **Research Skills:** A track record of completing end-to-end projects is the best signal. Papers are a great way to show this.
  - **Research taste**: The ability to come up with great research ideas *and* drive them to completion is rare and very valuable.
  - **Experiment design**: Can they design good experiments and make their research ideas concrete and convert them into actions?
- **Conceptual Understanding of Mech Interp:** Do you get the key ideas and know the literature?
- **Productivity and Conscientiousness:** This is a very hard one to interview for, but incredibly important. A public track record of doing interesting things is a good signal, as are strong references from trusted sources[36].
- **Engineering Skills:** Can you work fluently in a Python notebook? Can you write experiment code fast and well? Can you get things done? Do you understand the standard gotchas?
- **Deep engineering skill**: Beyond hacking together experiments, can you navigate large, complex codebases, write maintainable code, design complex software projects, etc?
  - This is much more important if doing research inside a larger lab or tech company than as an independent researcher or academic.
  - One of the most common reasons we don't hire seemingly promising researchers onto my team is because they lack sufficiently strong engineering skills.
  - Obviously, LLMs are substantially changing the game when it comes to engineering skills, but I think deep engineering skills will be much harder to

automate than shallow ones, unfortunately.
    - Unfortunately, I don't have great advice on how to gain these other than working in larger and more complex codebases and learning how to cope. Pair programming with more experienced programmers can be a great way to transfer tacit knowledge
- **Skepticism**: Can you constructively engage with research and critically evaluate it? In particular, can you do this to your own research? Good researchers need to be able to do work that is true.

Should you do a PhD?

I don't have a PhD (and think I would have had a far less successful career if I had tried to get one) so I'm somewhat biased. But it's a common question. Here are the strongest arguments I've heard in favour:

- You get extremely high **autonomy**. If you want to spend years going deep on a niche topic that no industry lab would fund, a PhD is one of the only ways to do it.
- It's a great environment to cultivate the ability to **set your own research agenda**. This is a crucial and difficult skill that is harder to learn in industry, where agendas are often set from the top down (though this varies a lot between team).

And here are the reasons I think it's often a bad idea:

- The opportunity cost is immense. You could spend 4-6 years gaining direct, relevant experience in an industry lab.
- Academic incentives can be misaligned with doing impactful research, e.g. pressure to publish meaning you're discouraged from admitting to the limitations of your work.
- The quality of supervision varies wildly, and a bad supervisor can make your life miserable.
- Quality of life: The pay is generally terrible, which may or may not matter to you, and you may only get places in a different city/country than you'd prefer.

But with all those caveats in mind, it's definitely the right option for some! My overall take:

- The key thing that matters is mentorship, being in an environment where you are working with a better researcher, and learning from them.
    - PhDs are often a good way of getting this. But if you can gain this by another way, plausibly you should go to that instead. PhDs have a lot of downsides too.

- Generally, the variance between supervisors and between managers in industry will dominate the academia versus industry differences, and thus you should pay a lot of attention to who exactly would be managing you.
    - For a PhD, try to speak to your potential supervisor's students in a private setting. If they say pretty bad things, that's a good reason not to go for the supervisor.
    - A common mistake is optimising for the most prestigious and famous supervisor when you often want to go for the ones who will have the most time for you, which anti-correlates.
- A common mistake is people feeling they need to *finish* PhDs. But if you sincerely believe that the point of a PhD is to be a learning environment, then why would the formal end of the PhD be the optimal time to leave? It's all kind of arbitrary.
    - IMO, at least every six months, you should seriously evaluate what other opportunities you have, try applying for some things and be emotionally willing leave if a better opportunity comes along (taking into account switching costs).
        - Note that often you can just take a year's leave of absence and resume at will.

Relevant Academic Labs

I'm a big fan of the work coming out of these two, they seem like great places to work:

- David Bau (Northeastern)
- Martin Wattenberg & Fernanda Viegas (Harvard)

Other labs that seem like good places to do interpretability research (note that this is not trying to be a comprehensive list!):

- Yonatan Belinkov (Technion)
- Jacob Andreas (MIT)
- Jacob Steinhardt (Berkeley)
- Ellie Pavlick (Brown)
- Victor Veitch (UChicago)
- Robert West (EPFL)
- Roger Grosse (Toronto)
- Mor Geva (Tel Aviv)
- Sarah Wiegreffe (Maryland)

- Aaron Mueller (Boston University)

Thanks a lot to Arthur Conmy, Paul Bogdan, Bilal Chughtai, Julian Minder, Callum McDougall, Josh Engels, Clement Dumas, Bart Bussmann for valuable feedback

---

1. ^ Note that I mean a full working month here. So something like 200 working hours. If you're only able to do this part-time, it's fine to take longer. If you're really focused on it, or have a head-start, then move on faster.
2. ^ If you want something even more approachable, one of my past MATS scholars recommends getting GPT-5 thinking to produce coding exercises (eg a Python script with empty functions, and good tests), for an easier way in.
3. ^ It's fine for this coding to need a bunch of LLM help and documentation/tutorial looking up, this isn't a memory test. The key thing is being able to correctly explain the core of each technique to a friend/LLM.
4. ^ Note: This curriculum aims to get you started on *independent research*. This is often good enough for academic labs, but the engineering bar for most industry labs is significantly higher, as you'll need to work in a large complex codebase with hundreds of other researchers. But those skills take much longer to gain.
5. ^ You want to exclude the first token of the prompt when collecting activations, it's a weird attention sink and often has high norm/is anomalous in many ways
6. ^ Gotcha: Remember to try a bunch of coefficients for the vector when adding it. This is a crucial hyper-parameter and steered model behaviour varies a lot depending on its value
7. ^ Mixture of expert models, where there are many parameters, and only a fraction light up for each token, are a pain for interpretability research. Larger models means you'll need to get more/larger GPUs which is expensive and unwieldy. Favor working with dense models where possible.
8. ^ You can download then upload the PDF to the model, or just select all and copy and paste from the PDF to the chat window. No need to correct the formatting issues, LLMs are great at ignoring weird formatting artifacts
9. ^ repo2txt.com is a useful tool for concatenating a Github repo into a single txt file
10. ^ If you would like other perspectives, check out Open Problems in Mechanistic Interpretability (broad lit review from many leading researchers, recent), or Interpretability Dreams (from Anthropic, 2 years old)

11. ^ And for reasons we'll discuss later, now feel much more pessimistic about the ambitious reverse engineering direction

12. ^ Even if you already have a research background in another field, mechanistic interpretability is sufficiently different that you should expect to need to relearn at least some of your instincts. This stage remains very relevant to you, though you can hopefully learn faster.

13. ^ The rest of this piece will be framed around approaching learning research like this and why I think it is a reasonable process. Obviously, there is not one true correct way to learn research! When I e.g. critique something as a "mistake", interpret this as "I often see people do this and think it's suboptimal for them", not "there does not exist a way of learning research where this is a good idea

14. ^ My term for associated knowledge, understanding, intuition, etc.

15. ^ Read my thoughts on SAEs here°. There's still useful work to be done, but it's an oversubscribed area, and our bar should be higher. They are a useful tool, but not as promising as I once hoped.

16. ^ This was using a technique called synthetic document fine-tuning (and some other creativity on top), which basically lets you insert false beliefs into a model by generating a bunch of fictional documents where those beliefs are true and fine-tuning the model on them.

17. ^ We chose problems we're excited to see worked on, while trying to avoid fad-like dynamics

18. ^ Latents refer to the hidden units of the SAE. These were originally termed "features", but that term is also used to mean "the interpretable concept the latent refers to", so I use a different term to minimise confusion.

19. ^ One of my MATS scholars make a working GPT-5 model diffing agent in a day

20. ^ This is the one line in the post *without* a "as of early Sept 2025" disclaimer, this feels pretty evergreen

21. ^ Note: "think" or "chain of thought" are terrible terms. It's far more useful to think of the chain of thought as a scratchpad that a model with very limited short-term memory can choose to use or ignore.

22. ^ Reasoning models break a lot of standard interpretability techniques because now the computational graph goes through the discrete, non-differentiable, and random operation of sampling thousands of times. Most interpretability techniques focus on studying a single forward pass.

23. ^ Not just, e.g., ones you can publish on.

24. ^ I called this moving fast in the blog post, but I think that may have confused some people.

25. ^ Though often this is done well with just a good introduction

26. ^ And having a well-known researcher as co-author is not sufficient evidence to avoid this, alas. I'm sure at least one paper I've co-authored in the past year or two is substantially false

27. ^ It's strongly in your interests for people to build on your work because that makes your original work look better, in addition to being just pretty cool to see people engage deeply with your stuff.

28. ^ Note that deliberately reproducing work, or trying to demonstrate the past work is shoddy, is completely reasonable. You just need to not *accidentally* reinvent the wheel.

29. ^ This is generally a good thing to do regardless of whether you're focused on research taste or not!

30. ^ And, nowadays, LLM knowledge too I guess?

31. ^ Note that you'll need someone who's written several Arxiv papers to endorse you. cs.LG is the typical category for ML papers.

32. ^ Note that you can submit something to a workshop *and* to a conference, so long as the workshop is "non-archival"

33. ^ A conference paper is a fair bit more effort, and you generally want to be working with someone who understands the academic conventions and shibboleths and the various hoops you should be jumping through. But I think this can be a nice thing to aim for, especially if you're starting out and need credentials, though mech interp cares less about peer review than most academic subfields.

34. ^ See this NeurIPS experiment showing that half the spotlight papers would be rejected by an independent reviewing council

35. ^ This is one of the most valuable things I do for my MATS scholars, IMO.

36. ^ Unfortunately, standard reference culture, especially in the US, is to basically lie, and the amount of lying varies between contexts, rendering references mostly useless unless from a cultural context the hiring manager understands or ideally from people they know and trust. This is one of the reasons that doing AI safety mentoring programs like MATS can be extremely valuable, because often your mentor will know people who might then go on to hire you, which makes you a lower risk hire from their perspective.

Moderation Log

More from Neel Nanda

15	Why Did My Model Do That? Model Incrimin…	aditya singh, gersonkroiz, Senthoor…	3d	0
58	models have some pretty funny attractor sta…	aryaj, Senthooran Rajamanoharan, …	17d	0
36	It Is Reasonable To Research How To Use Model Internals In Traini…	Neel Nanda	22d	10

View more

Curated and popular this week

56	Are there lessons from high-reliability engineering for AGI safety…	Steven Byrnes	3d	3
72	The persona selection model	Sam Marks	6d	2

37 Schelling Goodness, and Shared Morality as a Goal Andrew_Critch 2d 27